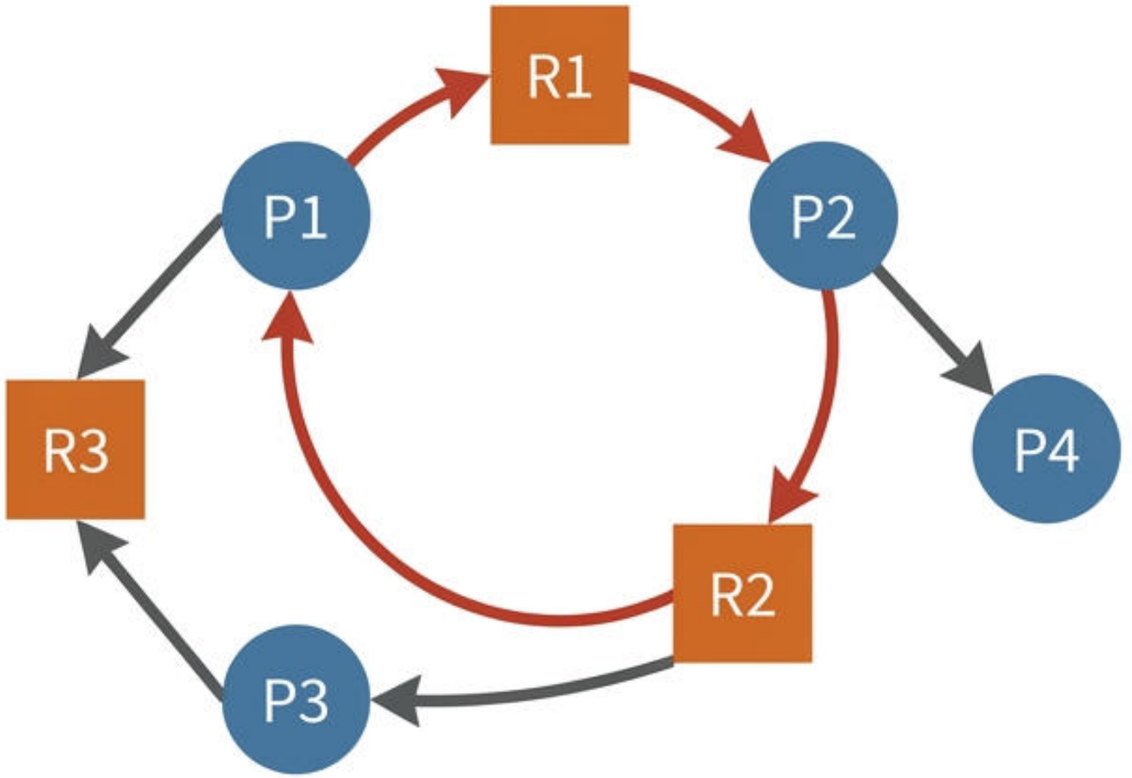
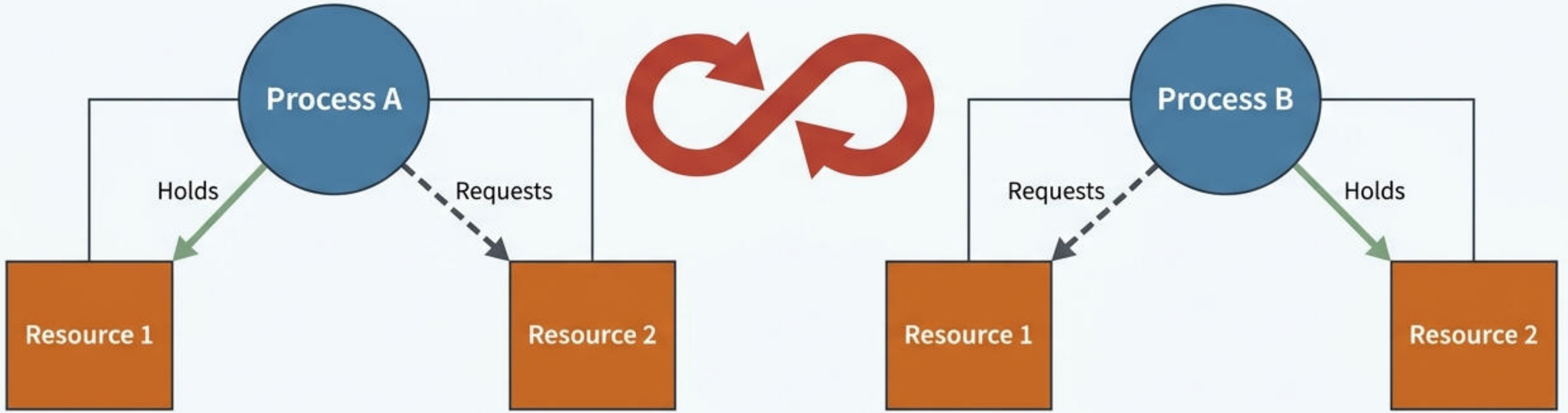


מערכות הפעלה: אנטומיה של קיפאון (Deadlock)

משאבים, מודלים, זיהוי, התחמקות ומניעה – מדריך חזותי מקיף.

דיאגרמת הקיפאון המעגלי: מצב ה'Mexican Standoff'



הגדרת הבעיה:

מצב שבו קבוצת תהליכים נאבקת על משאבים, וכל תהליך חסום משום שהוא ממתין לאירוע (שחרור משאב) שרק תהליך אחר באותה קבוצה יכול לגרום לו.

התוצאה הקלינית:

המתנה מעגלית אינסופית. אף תהליך לא מתקדם, מתקדם, ולכן אף משאב לא משוחרר.
המערכת קופאת לחלוטין.

סיווג משאבים ואזורי סכנה



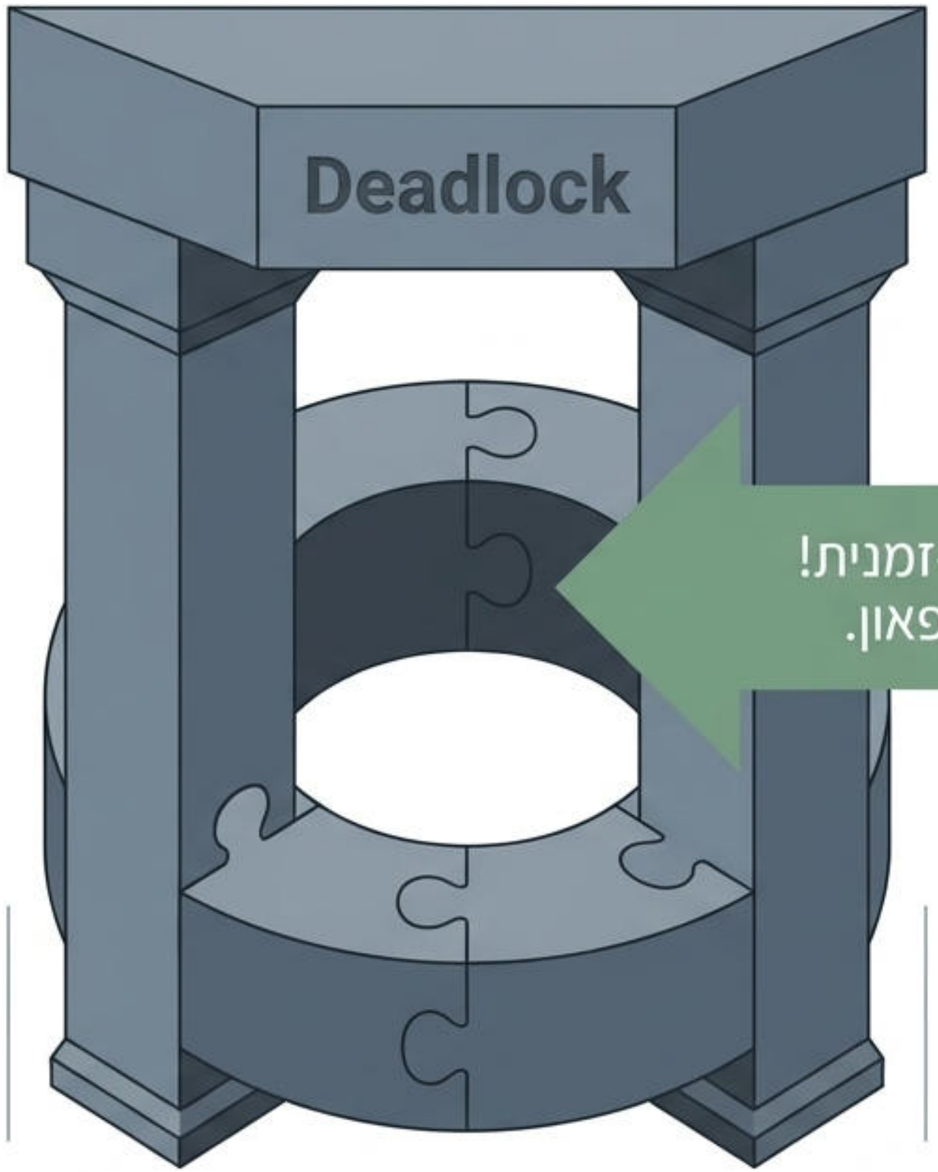


מניעה הדדית (Mutual Exclusion) –
לפחות משאב אחד במערכת מוחזק במצב בלעדי.

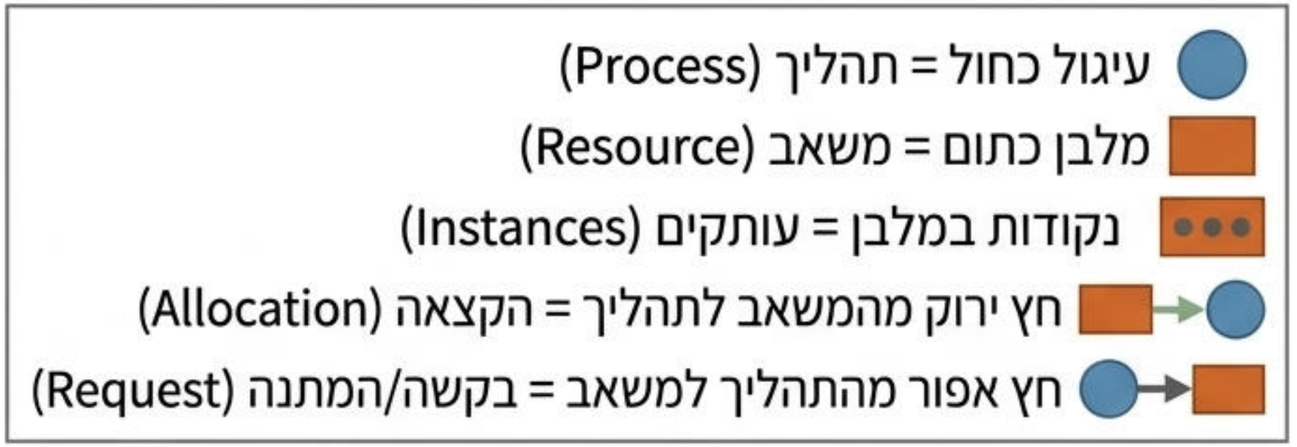
החזק והמתן (Hold and Wait) –
תהליך מחזיק משאב אחד לפחות, וממתין למשאב נוסף התפוס על ידי אחר.

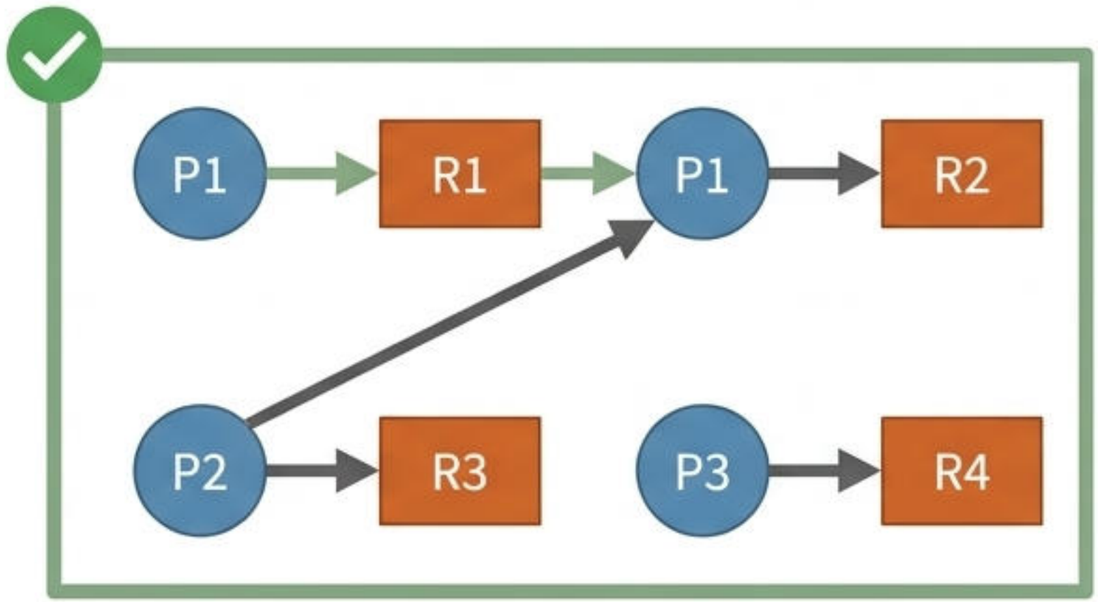
אי-חילוץ (No Preemption) –
לא ניתן להפקיע משאב בכוח, הוא ישוחרר רק מרצון מרצון התהליך.

המתנה מעגלית (Circular Wait) –
קיימת שרשרת סגורה של תהליכים שכל אחד ממתין לזה שאחריו.

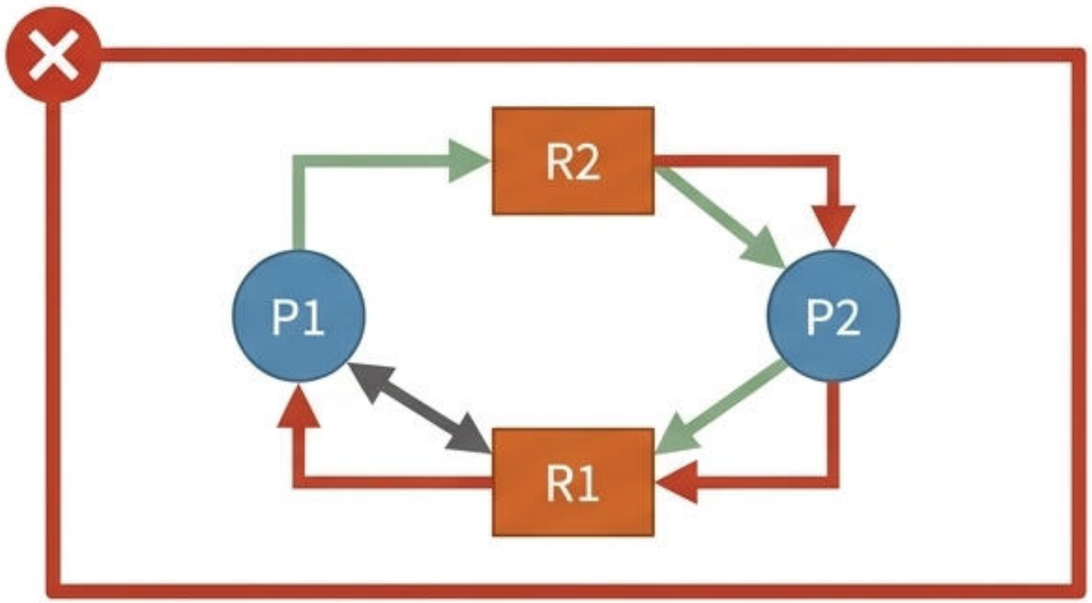


כל הארבעה חייבים להתקיים בו-זמנית!
שבירת תנאי אחד = חסינות מקיפאון.





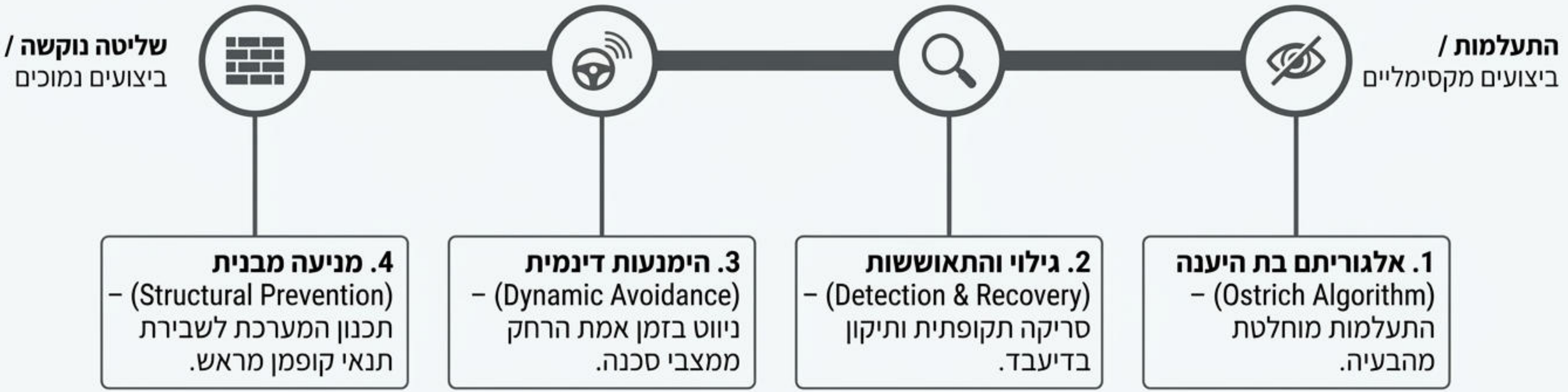
מערכת תקינה



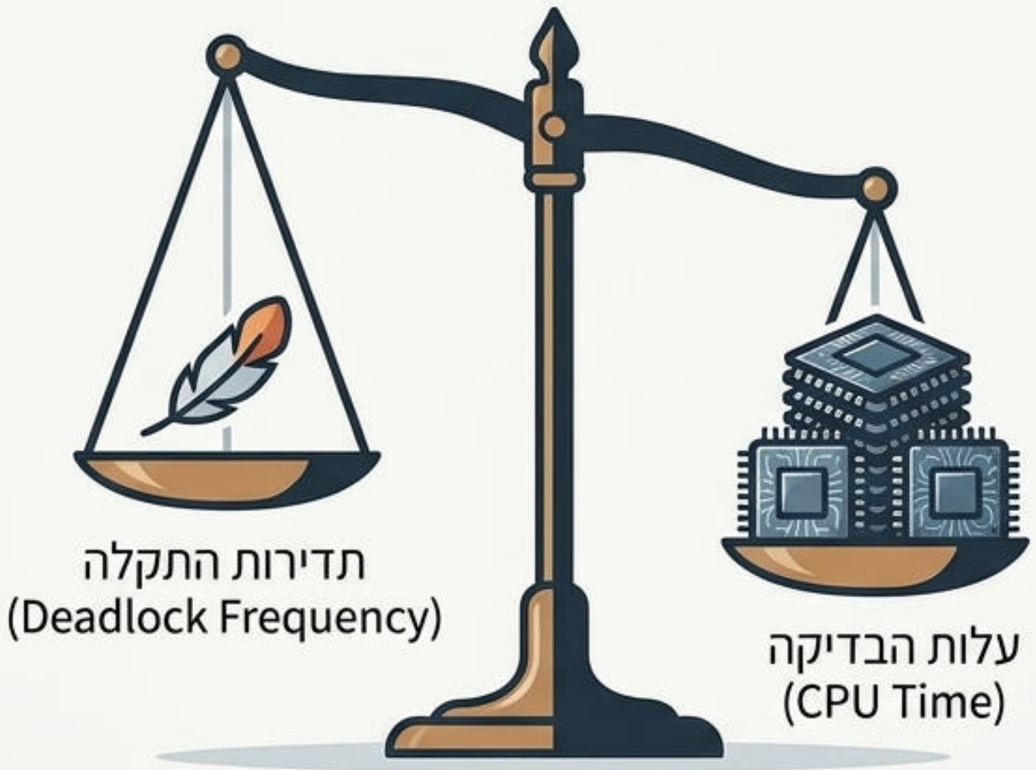
קיפאון ודאי

כלל הזיהוי (בעותק יחיד): מעגל סגור בגרף = הוכחה חותכת לקיפאון.
מסלול שחוזר לנקודת ההתחלה משמעותו שהמערכת תקועה.

Diagnostic Manual







הקונספט: פשוט להתעלם

אם קורה קיפאון אחת לתקופה ארוכה, המשתמש יבצע Reset או יסגור את התוכנה בכוח.
זוהי פשרה הנדסית לגיטימית.

ההיגיון: בדיקה מתמטית בזמן אמת

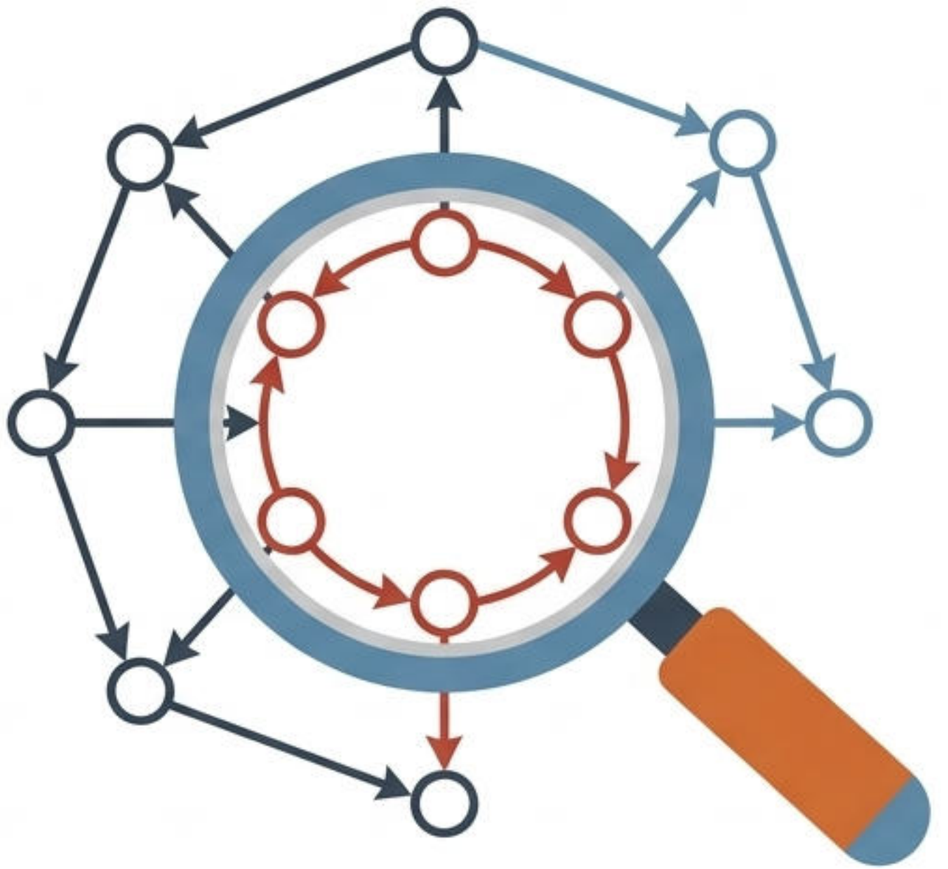
של כל בקשת משאב תאט את המחשב לכולם באופן קבוע.

במערכות הפעלה כלליות (Windows, Linux), המחיר לא שווה את התועלת.

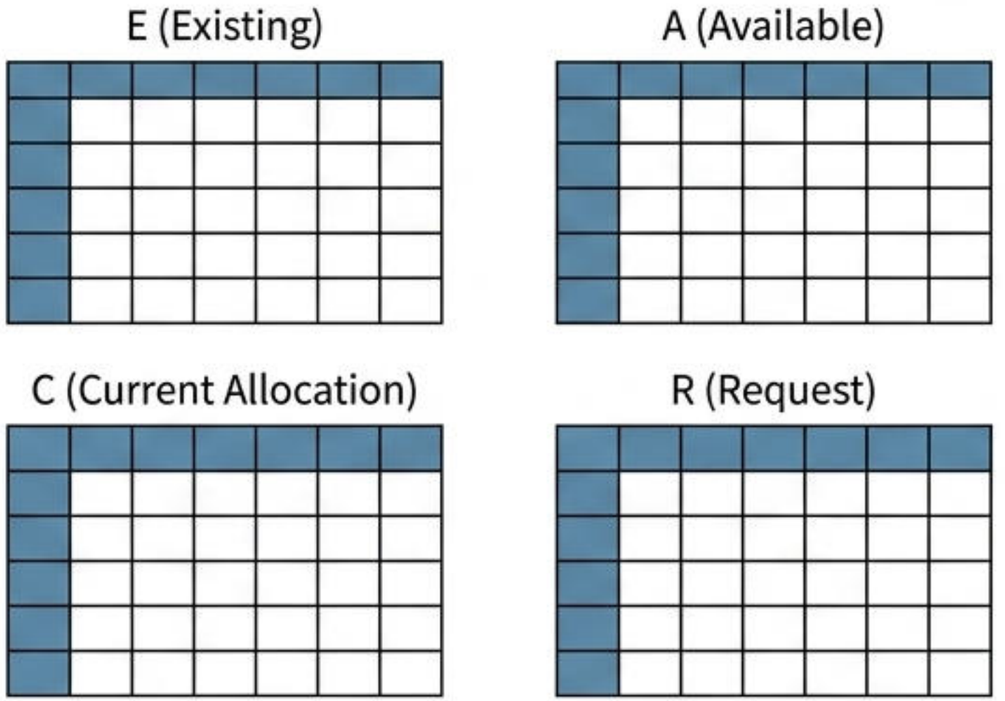
חריג קריטי: מערכות זמן אמת (Real-Time)

שם קריסה אינה באה בחשבון, וגישה זו פסולה.

עותק יחיד: חיפוש מעגל בגרף (DFS)



ריבוי עותקים: אלגוריתם הרדוקציה



אלגוריתם הרדוקציה:

1. חפש תהליך שכל בקשותיו (R) קטנות או שוות למלאי הפנוי (A).
2. הנח שתתהליך ירוץ, יסיים, ויחזיר את כל משאביו (C). הוסף משאבים אלו ל-A.
3. חזור על התהליך. כל תהליך שנותר ולא 'סיים' וירטואלית – נמצא בקיפאון.



חילוץ משאבים (Preemption)

הפקעת המשאב בכוח והעברתו לתהליך אחר. חיסרון: קשה לביצוע בחומרה (כמו מדפסת) מבלי להשחית נתונים.



חזרה לאחור (Rollback)

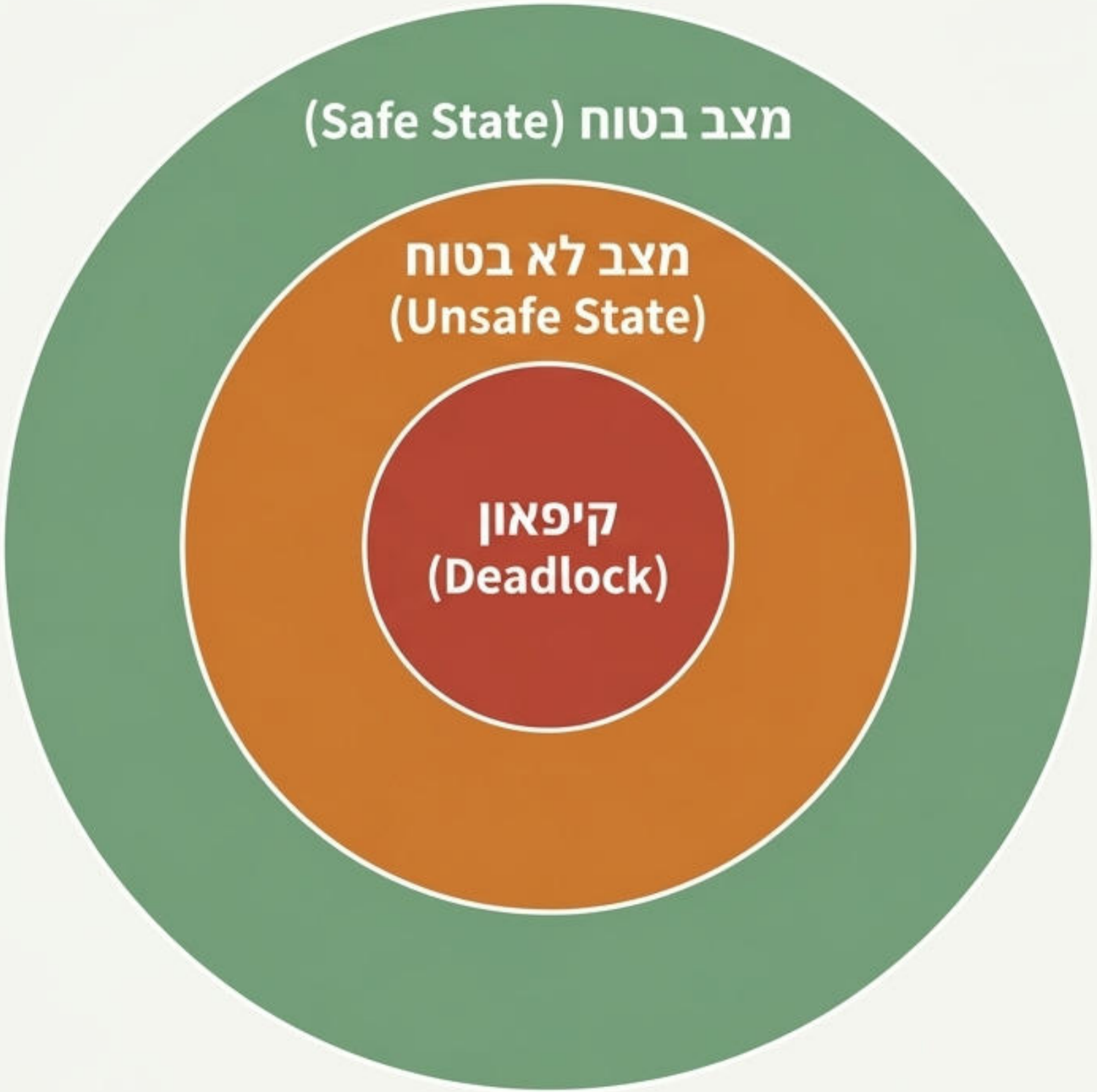
שימוש בנקודות שמירה תקופתיות (Checkpoints) וגלגול התהליך לאחור, לנקודה שבה טרם ביקש את המשאב.



חיסול תהליכים (Killing/Abort)

הריגת 'קורבן' ושחרור כל משאביו.

מניעת הרעבה (Starvation): המערכת מחשבת 'פונקציית עלות' בבחירת הקורבן, הכוללת את מספר הפעמים שהתהליך כבר חוסל בעבר, כדי למנוע את השמדתו הקבועה.



דרישת סף

תהליכים חייבים להצהיר מראש על כמות המשאבים המקסימלית שידרשו אי פעם (MAX).

מצב בטוח

קיים סדר הרצה כלשהו שיאפשר לכל התהליכים לסיים, גם אם כולם יבקשו כעת את המקסימום שלהם.

מצב לא בטוח

לא מובטח סדר הרצה בטוח. זהו אינו קיפאון מידי, אך המערכת מיידי, אך המערכת איבדה שליטה ואם כולם ידרשו מקסימום – קיפאון יהיה בלתי נמנע. המערכת דוחה כל בקשה שמובילה לכאן.

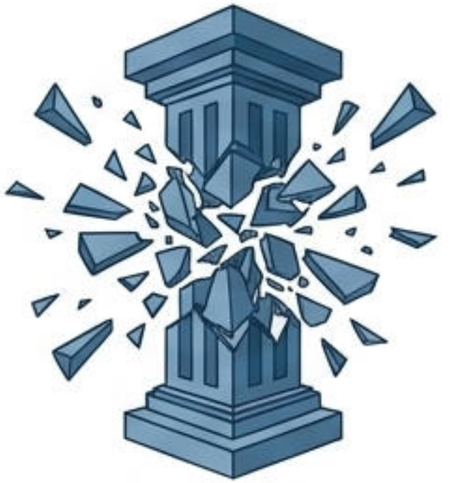
ארגז החול הווירטואלי (The Virtual Sandbox)



החיסרון המעשי: כמעט ולא מיושם בפועל. תוכניות מודרניות מתקשות לנבא ולהצהיר מראש את דרישות המקסימום שלהן, כיוון שהן תלויות קלט משתנה.

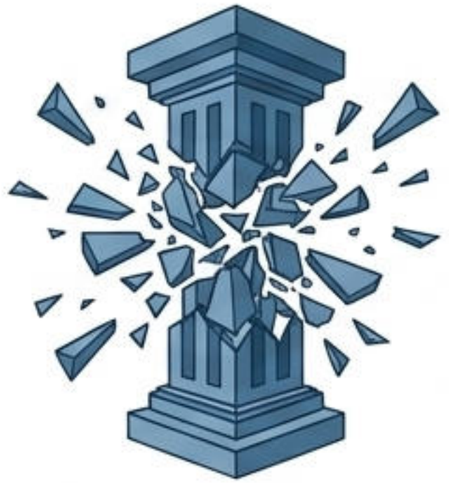
שבירת תנאי קיפאון

שבירת מניעה הדדית



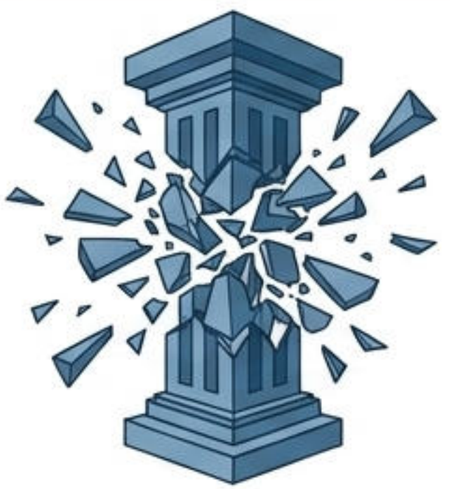
שימוש ב-Spooling.
הפיכת משאב בלעדי (מדפסת) לבר-שיתוף דרך תהליך מנהל (Daemon) אחד.

שבירת החזק והמתן



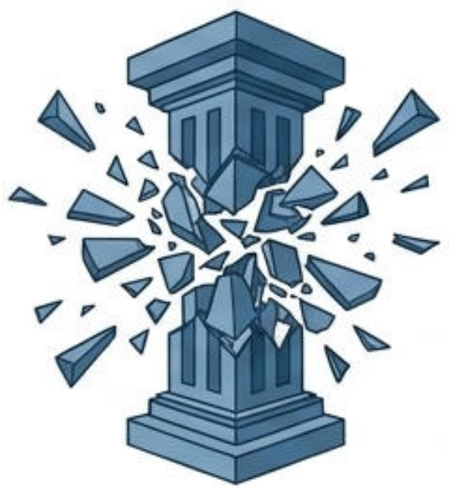
הקצאת 'הכל או כלום' (בקשת כל המשאבים מראש) או חובת שחרור כל המשאבים בטרם בקשת משאב חדש.

שבירת אי-חילוץ



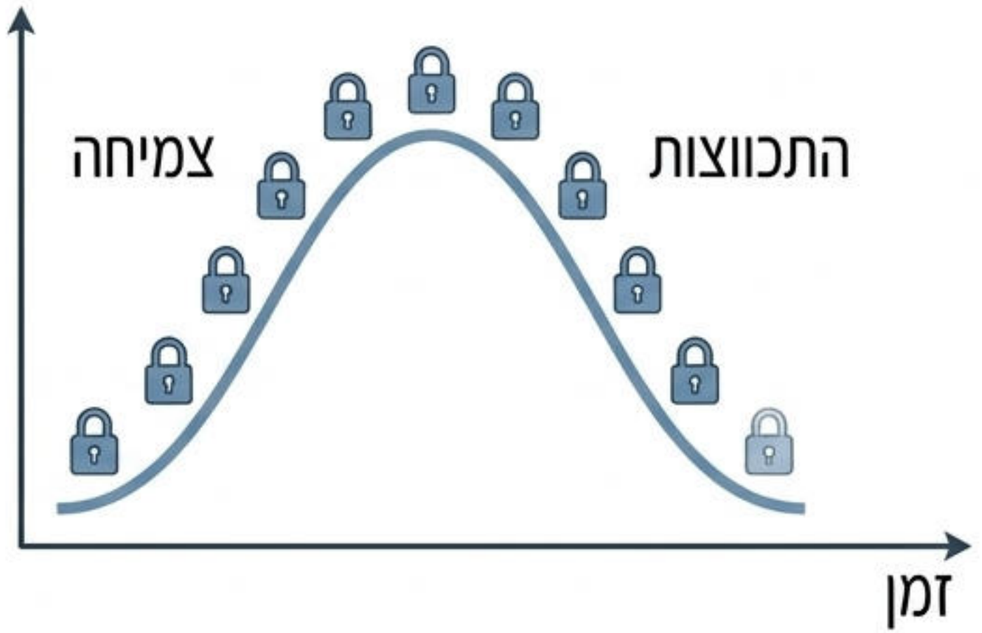
שימוש במשאבים וירטואליים.
הפקעה קלה באמצעות Switch Context במעבד או Swapping בזיכרון.

שבירת המתנה מעגלית



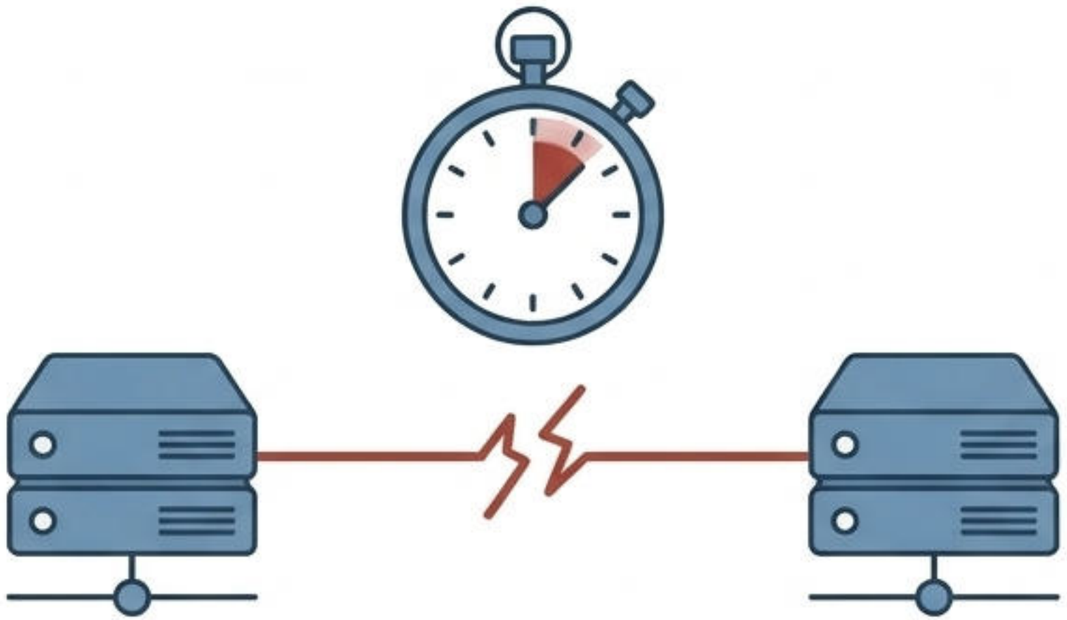
מספור היררכי. כל משאב מקבל מספר. חוק ברזל: תהליך מורשה לבקש משאבים אך ורק בסדר מספרי עולה.
לא ניתן 'לחזור אחורה' ולסגור מעגל.

נעילה דו-שלבית (מסדי נתונים)



תוקפת את 'החזק והמתן'.
שלב צמיחה: איסוף מנעולים. אם נתקל בחסימה, משחרר מיד את כולם, ממתין ומנסה שוב.
שלב התכווצות: ביצוע ושחרור ללא בקשות חדשות.

קיפאון תקשורת (רשתות)

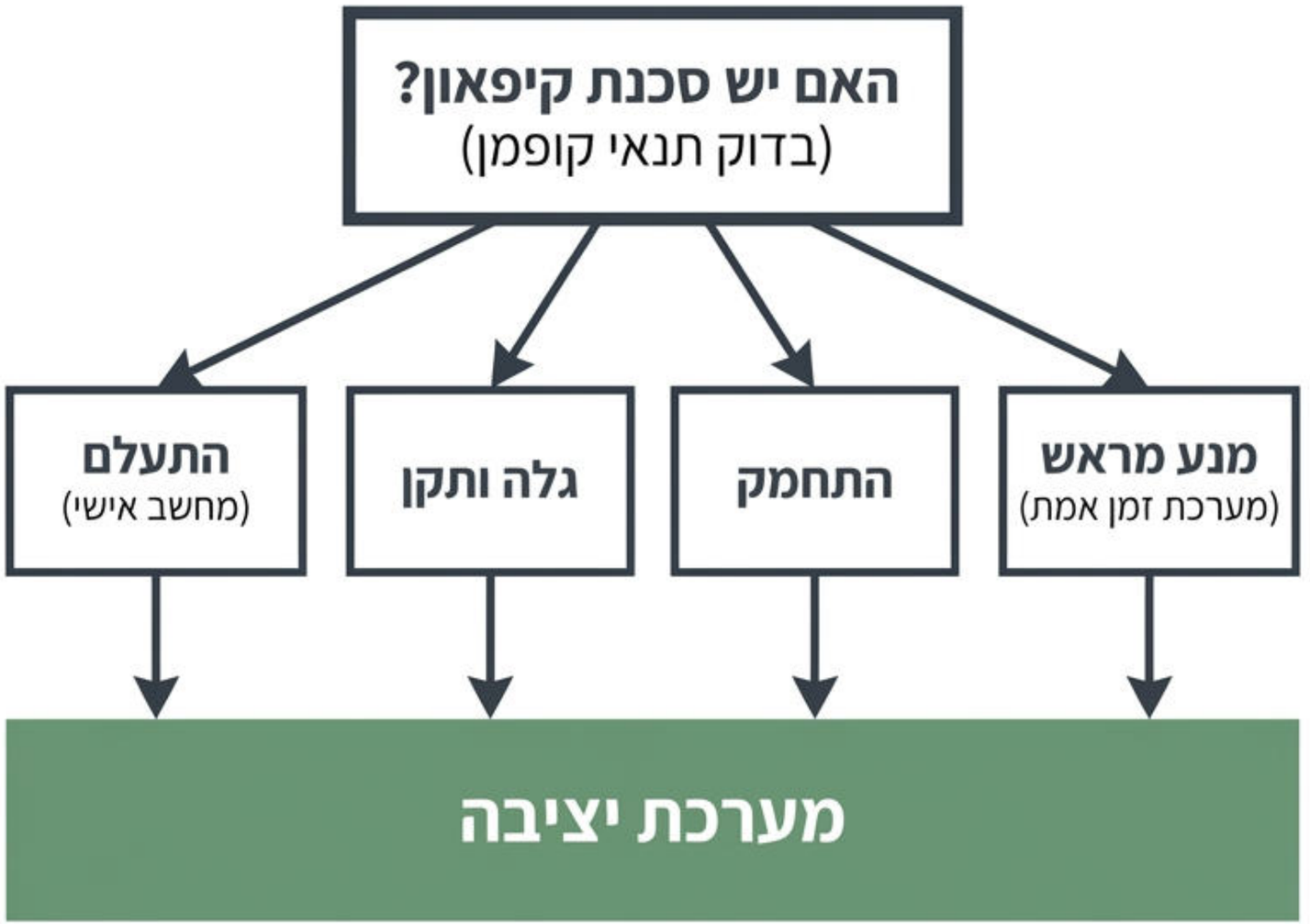


תהליך א' שלח בקשה וממתין, תהליך ב' ממתין לבקשה **(שאבדה ברשת)**. שניהם חסומים.
הפתרון: פסק זמן (Timeout).
אם אין אישור, משדרים מחדש.

תופעה (Phenomenon)	צריכת מעבד (CPU Usage)	מצב תהליכים (Process State)	הפתרון (The Fix)
קיפאון (Deadlock)	אפסית (0%)	חסומים, 'ישנים', ול ללא שינוי מצב	4 **אסטרטגיות** (מניעה, התחמקות וכו')
קיפאון חי (Livelock)	גבוהה (100% שריפת מעבד לריק)	אקטיביים, משנים מצב (סנכרון הרסני, כמו 2 אנשים במסדרון)	שבירת סימטריה המתנה **אקראית** (Random Backoff)
הרעבה (Starvation)	משתנה	המערכת בריאה, תהליך יחיד סובל מאפליה נצחית באלגוריתם התזמון	**התיישנות** (Aging) - העלאה הדרגתית של עדיפות התהליך הממתין

תהליך אבחון אדריכלי:

1. בדוק תנאי קופמן (האם קיים פוטנציאל?).
2. בחר אסטרטגיה מותאמת.



השורה התחתונה: עיצוב מערכות הפעלה הוא אומנות של פשרות. מהנדס המערכת מאזן תמיד בין ביצועים מקסימליים, ניצול מלא של חוומרה, ויציבות ללא פשרות.